

Common ways to access CSI data

- ESP32 and Raspberry Pi are the most common open platforms that support CSI. Most projects focus on these.
- Arduino boards do not provide CSI support.
- Qualcomm and Broadcom chips may support CSI but require specific laptop NICs and a Linux setup (e.g., Ubuntu 16.04 with a WLAN library).
- Intel 5300 NICs can extract CSI, but need compatible cards, custom firmware, and specialised tools. Setup is complex, and hardware is hard to source.
- Software-defined radios (SDRs) such as HackRF can capture physical-layer signals but not standard Wi-Fi CSI.
- Some high-end routers may expose physical-layer data; users can check documentation or flash custom firmware (e.g., OpenWRT), but this requires care.
- The Adafruit HUZZAH32 is an ESP32-based board, so it can support CSI extraction in theory.

CSI is usually available to network vendors to track how data packets move and whether they are received correctly. CSI provides information about the signal's amplitude and phase, showing how it evolves.

When a transmitter sends a Wi-Fi signal and a receiver captures it after it passes through a person, the signal's changes can be analysed further. Raw signals are filtered to remove irrelevant information, such as stationary objects in the room. Statistical methods isolate the parts of the signal that correspond to vital signs.

The filtered signal is then passed through a long short-term memory (LSTM) model, which identifies patterns corresponding to different heart rates. For example, the model learns the signal pattern at 60 beats per minute versus 110 beats per minute. By training the system on multiple people with varied heart rates, the model generalises to estimate heart rates

accurately for new users.

A training dataset can be sourced from third-party surveys or generated manually using an oximeter. However, if the ground truth is obtained from an oximeter, any inherent bias in its readings is propagated to the system. A bias of around 1 BPM and an error below 5 BPM are generally acceptable, as most applications track trends rather than exact values. A sudden change in heart rate, such as during dizziness or panic, is more significant than the precise numerical value. Even with sensor bias, the system can still detect functional patterns.

Turning Wi-Fi noise into data

Once the CSI data is collected, it must be cleaned before use. This is done using digital signal processing (DSP) techniques to extract useful information from raw Wi-Fi signals. The process starts by converting the complex Wi-Fi signal into

amplitude values. Each signal, originally in complex form, is transformed using a basic formula to obtain its amplitude.

After amplitude extraction, several filtering steps are applied. First, static interference, or DC noise, is removed from the signal. This ensures that the system focuses on dynamic physiological signals rather than environmental noise. Next, a Butterworth band-pass filter isolates the target signals within a specific range. The range is 48-130 BPM and can be adjusted for different applications, such as fall detection or monitoring higher heart rates. Finally, a Savitzky-Golay filter can be used to smooth the signal and remove remaining environmental noise.

Choosing the right filters and tuning the machine-learning model involves careful trial and error. Early filters may fail to classify the data correctly, so different combinations need to be tested. The LSTM model also requires tuning of its hyperparameters. Sometimes the model may appear to work well, but only because it is memorising the data. Using dropout and multiple training seeds helps reduce this overfitting. The final model should then be tested on new real-time data to ensure that it generalises reliably.

ML pipeline for heart-rate prediction

After filtering, the cleaned Wi-Fi signal is fed into an ML model for

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU